

GruaHub

Diagnóstico Executivo do Projeto

Visão para sócios: o que temos, o que falta, o que não precisamos, hardware (Adaptador Fino), integrações, instruções e próximos passos.

Data: 27 de julho de 2026

Referência: master/develop @ 43493c7 (release Adaptador Fino v0)

Confidencial — uso interno

Mensagem central

O sistema cloud é o centro operacional. O hardware é um adaptador fino e substituível — não uma placa all-in-one que amarra o cliente (modelo PagPlush).

1. Sumário executivo

O GruaHub é uma plataforma B2B multi-tenant para operadores de máquinas de pelúcia/gruas: frota, pagamento → crédito → jogada, conciliação, estoque, visitas de campo, finanças e portal do parceiro.

Estado em julho/2026

- Software MVP demonstrável ponta a ponta com simuladores.
- Firmware Pulse Adapter 0.1.0 compilando (PlatformIO / ESP32).
- Código em develop e master via gitflow (PRs #81 e #82).
- Falta: bancada física, piloto 3-10 máquinas, Pix real, TLS público.

Como ler este documento

Seções 3-5 respondem o que temos / não precisamos / falta. Seções 6-8 são instruções práticas. Seção 10 é o plano de ação dos sócios.

2. Posicionamento vs PagPlush

A PagPlush concentra operação numa PCB densa (ESP32, fonte, relé, optos, display, USB, bornes). Isso eleva custo, dificulta troca e gera lock-in.

Dimensão	PagPlush	GruaHub
Operação	Na placa	No cloud / SaaS
Hardware	All-in-one proprietário	DevKit + harness fino
Lock-in	HW + operação	SaaS; adaptador trocável
Field-swap	Troca o cérebro	Desplug → outro DevKit
Escala	SKU complexo	SKU fino + cabos

Decisão estratégica

Não competir em complexidade de PCB. Competir em uptime da frota, conciliação, custo de troca e centralização operacional no sistema.

3. O que TEMOS (ativos)

3.1 Software

- [x] Backend Quarkus 3.8 / Java 21 — monólito modular
- [x] Web Next.js 14 — ops, PWA jogador, portal parceiro
- [x] Mobile Expo — operador offline-first
- [x] Keycloak 24, EMQX MQTT, PostgreSQL, MinIO
- [x] Simuladores de máquina e pagamento
- [x] Contratos MQTT v1/ + OpenAPI versionados
- [x] Domínios: frota, IoT, pagamentos sandbox, jogadas, conciliação, estoque, visitas, sangria, comissão, alertas, anexos, push

3.2 Hardware (firmware / spec)

- [x] Spec Adaptador Fino v0 documentada

- [x] Firmware Pulse: heartbeat, GRANT_CREDIT idempotente, ACK, PLAY_IN opcional
- [x] Provisionamento NVS (Serial + script)
- [x] Build esp32dev SUCCESS

3.3 Engenharia e operação

- [x] Docker Compose + profiles prod-like / tls / backup / monitoring
- [x] scripts/ci-local.sh (contracts + simulators + firmware)
- [x] Gitflow feature → develop → release → master

4. O que NÃO precisamos (agora)

Anti-lista (proteger o produto)

Tudo abaixo é tentador sob pressão competitiva e deve ser rejeitado no piloto.

- PCB all-in-one com display, USB host, multi-relé e dezenas de optos
- Lógica de preço, promoção ou estoque no firmware
- Bornes de parafuso como interface principal de instalação
- Microserviços / Kubernetes / Kafka sem evidência de carga
- Carteira B2C, NF-e SEFAZ, IA/ML preditivo nesta fase
- Fabricar PCB própria antes de validar DevKit em campo

5. O que FALTA

P0 — bloqueia fruto comercial

- [] Bancada E2E com ESP32 físico + LED/relé
- [] 1 adquirente Pix real em tenant piloto
- [] Piloto 3-10 máquinas com harness
- [] DNS + TLS público (Let's Encrypt)
- [] Billing GitHub Actions (ou aceitar só ci-local)

P1 — portabilidade e escala

- [] Cabos harness por fabricante (SKU de cabo)
- [] Vendor adapter Eletek/Sega com protocolo real
- [] Provisionamento QR + bind mobile
- [] TLS MQTT + ACL por device
- [] Dashboard crédito / mismatch
- [] E-mail/SMS, WAF comercial, Vault/SM, Pitr

6. Hardware — placas e instruções

Comprar peças de prateleira. Não projetar PCB custom no piloto.

BOM recomendado

Item	Função	Nota
ESP32-WROOM ou C3 DevKit	MCU + Wi-Fi	envs esp32dev / esp32c3
Relé 5V ou opto PC817	Pulso crédito	GPIO 26 default
JST/Molex 2-4 vias + cabo	Harness	1 cabo por modelo de grua
Fonte 5V ≥ 1A	Alimentação	Buck 12→5 se necessário
LED	Prova de pulso	Onboard ok

Pinout do harness

1 GND | 2 CREDIT_OUT | 3 PLAY_IN (opc.) | 4 +5V (opc.)

Instalação alvo (≤ 15 minutos)

- 1. Criar/associar machine + controller no dashboard
- 2. Flash firmware e gravar NVS (tenant, machine, Wi-Fi, MQTT, pulseMs)
- 3. Conectar harness (mínimo GND + CREDIT_OUT)
- 4. Energizar → heartbeat → ONLINE
- 5. Disparar 1 crédito → observar pulso → ACK no backend

Build e provisionamento

```
cd firmware/adaptador-fino
pio run -e esp32dev && pio run -t upload
scripts/provision-adaptador-nvs.sh /caminho/lab.env
```

7. Integrações

Integração	Estado	Próximo passo
MQTT EMQX v1/	Pronto	TLS + ACL produção
Pix sandbox	Pronto	Adquirente real
Mercado Pago	Parcial	Token + tenant piloto
Eletek / Sega	Stubs	Após pulse estável
Expo Push	Pronto	FCM Google opcional
MinIO anexos	Pronto	Bucket produção
Keycloak	Pronto	Sem usuários demo

Fluxo canônico

Jogador/Pix → Backend → Outbox GRANT_CREDIT → MQTT
→ Adaptador (pulsos) → CREDIT_RECEIVED → Conciliação

8. Instruções do sistema (quickstart)

Demo local

```
cp infra/.env.example infra/.env
cd infra && ./run.sh up
cd backend && ./mvnw quarkus:dev
cd web && npm i && npm run dev
# + simulators/machine-simulator e payment-simulator
```

Validação local (sem Actions)

```
./scripts/ci-local.sh
# CI_LOCAL_JOBS=all ./scripts/ci-local.sh
```

Documentos no repositório

- README.md — visão e quickstart
- docs/HARDWARE_ADAPTER.md — hardware fino
- docs/MQTT_CONTRACT.md — contrato IoT
- docs/DEPLOYMENT.md — deploy / DR
- docs/COMMERCIAL_READINESS.md — gaps comerciais
- docs/PILOT_CHECKLIST.md — checklist piloto + bancada
- docs/ARCHITECTURE.md — arquitetura
- docs/TASKS.md — estado real das tasks

9. Arquitetura (resumo)

Clientes Web/Mobile/PWA autenticam via Keycloak. O backend Quarkus expõe REST e consome/publica MQTT via EMQX. Persistência em PostgreSQL; objetos em MinIO. O Adaptador Fino (ou simulador) é cliente MQTT fino: executa crédito e reporta estado.

```
Web / Mobile / PWA → Keycloak → Quarkus API
ESP32 / Simulador → EMQX MQTT → Quarkus IoT
Quarkus → PostgreSQL + MinIO + Outbox → comandos MQTT
```

10. Plano de ação para os sócios

Quando	Ação	Dono
Semana 1	Comprar 2-3 DevKits + relés; montar bancada	Técnico/ops
Semana 1-2	Contrato Pix (ex. Mercado Pago)	Comercial
Semana 2-3	E2E bancada + 1 máquina real	Técnico
Semana 3-6	Piloto 3-10 máquinas	Ambos
Paralelo	DNS + HTTPS público	Técnico
Depois	Vendor adapter se frota exigir	Técnico
Evitar	PCB monstro / feature creep no device	Ambos

11. Diagnóstico de riscos

Risco	Sev.	Mitigação
Sem hardware em campo	Alta	Bancada + piloto especificados
Sem Pix real	Alta	1 adquirente prioritário
CI Actions (billing)	Média	ci-local.sh
Copiar modelo PagPlush	Alta	Spec anti all-in-one
Wi-Fi ruim no ponto	Média	Antena; 4G só depois

12. Conclusão

A base de software está sólida e a tese de hardware está correta. Para gerar receita, o foco não é inventar placa complexa: é fechar o elo físico mínimo (DevKit + harness), ligar Pix real e rodar um piloto curto, mantendo o GruaHub como sistema central da operação.

Próxima revisão sugerida

Atualizar este PDF após: (1) bancada E2E OK, (2) primeiro Pix real, (3) primeiro piloto com máquinas reais.

Fonte regenerável: `scripts/generate-diagnostico-pdf.py`

Saídas: `docs/diagnostico/GruaHub-Diagnostico-Executivo.pdf`

`docs/diagnostico/GruaHub-Diagnostico-Executivo.md`

e